Lab 57: Keep discovery out of the main thread

Topic 5: Context Management & Reliability · Task 5.4: Manage context effectively in large codebase exploration

Runs on Simulator 6: Structured Data Extraction · Self-practice

Source: SkillCertPro practice exam Part 1, Q22 (unofficial third-party question)

The question

The team is adding a new document type. Claude Code must first inspect dozens of existing extractor modules, schema validators, fixture conventions, and error-handling paths. In trial runs, the main conversation fills with search and file-read output, and by implementation time it loses track of the target schema changes. Which workflow best preserves useful context while still doing discovery?

A. Switch the whole task to plan mode and let it inspect, edit, and validate files in one continuous context.

B. Delegate the exploratory search to the built-in Explore subagent with explicit scope and conventions, then implement from its summarized findings.

C. Create a shared slash command that imports all extractor, fixture, and schema files into the prompt before editing.

D. Run the searches in the main conversation, then use /compact immediately before implementing the schema and validator changes.

Simulate the solution

```
# In the browser: open Simulator 6, then Lab 57
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 57

# In a terminal, from simulators/simulator-6-structured-extraction/
uv run extract.py --case UC-22
```

What you should see

Prints the delegation prompt to paste into Claude Code. The main thread stays short.

evidence/cases/UC-22.json holds the rendered delegation prompt and the expected answer; the learner's main conversation stays short and the implementation step still knows the target schema changes.

Open `simulators/simulator-6-structured-extraction/evidence/` and find it yourself.

The answer

B. Explore subagent with explicit scope, implement from its summary.

Discovery produces lots of low-level evidence (file listings, grep hits, repeated reads), but implementation needs only the conclusions. The built-in Explore subagent does read-only discovery. It is invoked through the Agent tool, runs in a separate context window, and returns a single text result: the parent never sees its intermediate calls. Give it a focused delegation prompt (what to investigate, which schema and validation conventions matter, what format the summary should take), and restate project-specific rules, because it does not inherit the conversation.

Why the others fail

A. Plan mode is for reviewing an approach before edits. It does not keep verbose discovery out of the main context.

C. Importing every file floods the context even more.

D. Compacting after the fact is lossy, and the target-schema details may be summarised away.

Check yourself

- Learner explains why the delegation prompt restates the extraction conventions (Explore inherits no history).
- Learner distinguishes /compact, plan mode and subagent delegation by the problem each solves.
- Learner's prompt bounds the output (line cap, named sections).

Now change one thing

Change DOC-04's printed total in fixtures/invoices.txt, rerun, and watch by_document_type.invoice fall while the others hold.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.